

**LAPORAN TUGAS BESAR**  
**IF2224 – TEORI BAHASA FORMAL DAN OTOMATA**



K01 - MLB

|                            |          |
|----------------------------|----------|
| Mikhael Benrael Tampubolon | 13524009 |
| Muhammad Iqbal Raihan      | 13524011 |
| Moh Hafizh Irham Perdana   | 13524025 |
| Muhammad Akmal             | 13524099 |

**PROGRAM STUDI TEKNIK INFORMATIKA**  
**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**  
**INSTITUT TEKNOLOGI BANDUNG**  
**2025**

# DAFTAR ISI

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>DAFTAR ISI.....</b>                           | <b>2</b>  |
| <b>BAB 1 DASAR TEORI.....</b>                    | <b>4</b>  |
| 1.1. Deterministic Finite Automata (DFA).....    | 4         |
| 1.2. Lexical Analyzer (Lexer).....               | 5         |
| 1.3. Token dan Tokenization.....                 | 6         |
| <b>BAB 2 PERANCANGAN DAN IMPLEMENTASI.....</b>   | <b>8</b>  |
| 2.1. Perancangan Diagram Transisi DFA.....       | 8         |
| A. Sub-Diagram String dan Charcon.....           | 9         |
| B. Sub-Diagram Numbers (Intcon dan Realcon)..... | 10        |
| C. Sub-Diagram Comment.....                      | 11        |
| D. Sub-Diagram Operator dan Tanda Baca.....      | 12        |
| E. Sub-Diagram Keywords dan Identifier.....      | 14        |
| F. Accepting State ident.....                    | 15        |
| 2.2. Implementasi Program.....                   | 16        |
| A. Modul Klasifikasi Karakter.....               | 16        |
| B. Modul Representasi Token.....                 | 17        |
| C. Modul Pembaca Karakter.....                   | 18        |
| D. Modul State Machine.....                      | 19        |
| E. Modul Lexical Analyzer.....                   | 20        |
| <b>BAB 3 PENGUJIAN.....</b>                      | <b>22</b> |
| 3.1. Hasil Pengujian.....                        | 22        |
| A. Pengujian ke-1.....                           | 22        |
| B. Pengujian ke-2.....                           | 22        |
| C. Pengujian ke-3.....                           | 25        |
| D. Pengujian ke-4.....                           | 26        |
| E. Pengujian ke-5.....                           | 28        |
| F. Pengujian ke-6.....                           | 29        |
| G. Pengujian ke-7.....                           | 30        |
| H. Pengujian ke-8.....                           | 30        |
| I. Pengujian ke-9.....                           | 30        |
| J. Pengujian ke-10.....                          | 30        |
| <b>BAB 4 KESIMPULAN DAN SARAN.....</b>           | <b>32</b> |
| 4.1. Kesimpulan.....                             | 32        |
| 4.2. Saran.....                                  | 32        |
| <b>BAB 5 LAMPIRAN.....</b>                       | <b>33</b> |
| 5.1. Link Repository.....                        | 33        |
| 5.2. Link DFA.....                               | 33        |
| <b>REFERENSI.....</b>                            | <b>34</b> |

# BAB 1

## TEORI DASAR

### 1.1. Deterministic Finite Automata (DFA)

Deterministic Finite Automata (DFA) adalah model komputasi matematis yang digunakan untuk mengenali pola dalam suatu string masukan. DFA merupakan salah satu jenis *finite automaton* yang bersifat deterministik, artinya untuk setiap pasangan state dan simbol masukan, terdapat tepat satu transisi ke state berikutnya. Hal ini membedakan DFA dari NFA (*Nondeterministic Finite Automata*) yang memungkinkan lebih dari satu transisi untuk pasangan state dan simbol yang sama.

Secara formal, sebuah DFA didefinisikan sebagai 5-tuple  $(Q, \Sigma, \delta, q_0, F)$  yang terdiri dari:

- **Q**: Himpunan berhingga dari state.
- **$\Sigma$** : Himpunan berhingga simbol masukan yang disebut alfabet.
- **$\delta$** : Fungsi transisi  $\delta: Q \times \Sigma \rightarrow Q$  yang memetakan pasangan state dan simbol masukan ke state berikutnya.
- **$q_0$** : State awal (*initial state*), di mana  $q_0 \in Q$ .
- **F**: Himpunan *accepting states* atau *final states*, di mana  $F \subseteq Q$ .

DFA memproses string masukan karakter demi karakter dari kiri ke kanan. Dimulai dari state awal  $q_0$ , setiap karakter yang dibaca menyebabkan transisi ke state berikutnya sesuai fungsi transisi  $\delta$ . Setelah seluruh karakter habis dibaca, string diterima (*accepted*) jika dan hanya jika DFA berada pada salah satu state dalam F. Sebaliknya, jika DFA berakhir pada state yang bukan *accepting state*, string tersebut ditolak (*rejected*).

DFA dapat direpresentasikan secara visual melalui diagram transisi, yaitu graf berarah di mana setiap simpul (*node*) merepresentasikan sebuah state dan setiap sisi (*edge*) berlabel simbol merepresentasikan transisi antar state. State awal ditandai dengan anak panah masuk, sedangkan *accepting state* ditandai dengan lingkaran ganda.

Salah satu sifat penting DFA adalah kemampuannya untuk mengenali bahasa reguler (*regular language*), yaitu kelas bahasa yang dapat dideskripsikan menggunakan *regular expression*. Berdasarkan teorema Kleene, setiap bahasa reguler dapat dikenali

oleh suatu DFA, dan sebaliknya, setiap bahasa yang dikenali DFA adalah bahasa reguler. Sifat inilah yang menjadikan DFA sangat relevan dalam bidang pemrosesan bahasa, khususnya pada tahap analisis leksikal dalam proses kompilasi, di mana pola-pola token umumnya dapat dideskripsikan menggunakan *regular expression* yang kemudian diimplementasikan sebagai DFA.

## 1.2. Lexical Analyzer (Lexer)

Lexical Analysis atau analisis leksikal merupakan fase pertama dalam sebuah *compiler*. Tahap ini bertugas mengubah *source code* atau sejumlah input karakter dari sumber file ke dalam sebuah urutan dengan memisah-misah input ke dalam beberapa bagian dan melewati karakter/input yang tidak relevan agar kode dapat dipahami dengan lebih mudah oleh mesin. Aktor yang melakukan hal ini disebut sebagai *lexical analyzer*. Lexical Analyzer akan membaca *source code* dari kiri ke kanan, kemudian mengelompokkan input-input yang berkaitan menjadi unit yang disebut lexeme. Lexeme kemudian dikelompokkan menjadi token, yaitu tipe atau kelas tertentu seperti INTEGER, IDENTIFIER, IF, WHILE, dan REAL.

Selain sebagai pengubah karakter menjadi token, analisis leksikal memberikan beberapa keuntungan krusial bagi proses selanjutnya yaitu,

- a. Menyederhanakan analisis sintaks dengan membuang detail yang tidak relevan seperti *whitespace* (spasi, tab) dan komentar sehingga beban kerja *parser* menjadi jauh lebih ringan karena tidak perlu menangani karakter kosong.
- b. Mengompresi input sehingga data lebih mudah masuk ke dalam cache mesin dan mempercepat proses identifikasi
- c. Melacak koordinat pada *source code* karena Lexer menyimpan informasi setiap token, termasuk nama file, nomor baris (line number), dan indeks karakter. Data ini sangat penting untuk menandai letak *error* secara akurat dan melacak di mana sebuah simbol didefinisikan.
- d. Menyimpan nilai asosiasi untuk token seperti konstanta atau *identifier*. Lexer tidak hanya mengidentifikasi tipenya, tetapi menyimpan nilai seperti nilai numerik dari konstanta integer ke dalam tabel simbol.

Agar Lexer dapat mengenali input, kita dapat menggunakan sebuah barisan regex (regular expressions) bersamaan dengan tipe token yang dikenali. Namun, regex terkadang bersifat ambigu (satu input cocok dengan lebih dari satu aturan) sehingga digunakan dua prinsip utama dalam pemilihan token:

(1) Longest Possible Match

Lexer akan mengambil urutan karakter terpanjang yang mungkin yang sesuai pola. Sebagai contoh, input `ifxyz` akan dianggap sebagai satu IDENTIFIER, bukan kata kunci `IF` diikuti identifier `xyz`.

(2) Rule Priority

Aturan ini berlaku apabila masih terdapat beberapa aturan regex yang cocok dengan panjang karakter yang sama. Aturan yang didefinisikan lebih awal akan diambil di atas aturan umum lain.

### 1.3. Token dan Tokenization

Token merupakan unit leksikal dasar dari sebuah bahasa pemrograman yang dihasilkan oleh *lexical analyzer* setelah membaca dan memproses teks dari kode sumber. Pembuatan token bertujuan untuk mengabstraksikan dan mengklasifikasikan detail input menjadi kategori-kategori yang lebih ringkas, sehingga menyederhanakan pekerjaan pada fase analisis sintaksis (*parser*).

Untuk setiap token yang dikenali, Lexical Analyzer mengembalikan dua komponen utama informasi:

- Kode Token (*Token Code*): Representasi utama dari kelas token tersebut. Untuk token karakter tunggal seperti operator aritmatika (`*`) atau tanda hubung (`=`), kode tokennya adalah nilai numerik dari karakter itu sendiri. Sedangkan untuk token multikarakter, digunakan konstanta enumerasi yang telah didefinisikan, seperti `ID` untuk identifier, `ICON` untuk konstanta integer, atau `IF` untuk kata kunci.
- Nilai Asosiasi (*Associated Values*): Data spesifik yang terikat pada suatu token. Jika kode token memberikan "jenis" token, nilai asosiasi memberikan "isi" sebenarnya. Sebagai contoh, untuk ekspresi `ptr`, *lexer* akan mengembalikan kode `ID` beserta nilai asosiasi berupa string `"ptr"` dan pointer ke entri tabel simbol

(*symbol-table entry*) dari identifier tersebut. Sebaliknya, token struktural seperti operator umumnya tidak memiliki nilai asosiasi.

Terdapat dua metode utama untuk mengenali token yaitu dengan membuat Finite Automaton (FA) atau menuliskannya secara manual (ad hoc). Struktur leksikal dari sebagian besar bahasa pemrograman dapat dideskripsikan melalui Regular Expression yang merupakan ekspresi yang dapat digunakan untuk membangun DFA (Deterministic Finite Automata) yang akan mengenali dan memberikan sebuah token. Kelebihan dari pendekatan ini adalah pengenalan token dapat dilakukan secara otomatis.

Implementasi penggunaan DFA dalam mengenali token didasari dari definisi berikut:

**Definisi 1.** Sebuah FA (*finite automaton*) dari sebuah alfabet berhingga  $\Sigma$  terdiri dari

- Himpunan berhingga  $Q$  dari *state* (keadaan).
- Himpunan  $\Delta \subseteq Q \times \Sigma \times Q$  dari sisi (*edges*) dari suatu *state* ke *state* lainnya yang diberi label huruf-huruf dari alfabet input  $\Sigma$ . FA juga menerima  $\epsilon$  sebagai label pada sebuah sisi, yang berarti bahwa  $(q, \epsilon, q)$  adalah transisi spontan dari  $q$  ke  $q$  yang tidak mengonsumsi input apapun.
- Sebuah *initial state* (keadaan awal)  $q_0 \in Q$ .
- Himpunan dari *accepting states*  $F \subseteq Q$ .

DFA tersebut menerima input *string*  $w = a_1 a_2 \dots a_k \in \Sigma^*$  jika dan hanya jika terdapat  $n \in \mathbb{N}$  dan urutan *state*  $q_0, q_1, q_2, \dots, q_n \in Q$  di mana  $q_0$  adalah *initial state* dan  $q_n \in F$  adalah *accepting state*, sedemikian sehingga  $(q_{i-1}, a_i, q_i) \in \Delta$  untuk semua  $i = 1, 2, \dots, n$ . Secara visual, kondisi tersebut bersesuaian dengan keberadaan sekumpulan sisi dalam FA yang diberi label oleh input yang sesuai:

$$q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} q_2 \xrightarrow{a_3} q_3 \xrightarrow{a_4} \dots \xrightarrow{a_{n-1}} q_{n-1} \xrightarrow{a_n} q_n \in F$$

## BAB 2

# PERANCANGAN DAN IMPLEMENTASI

### 2.1. Perancangan Diagram Transisi DFA

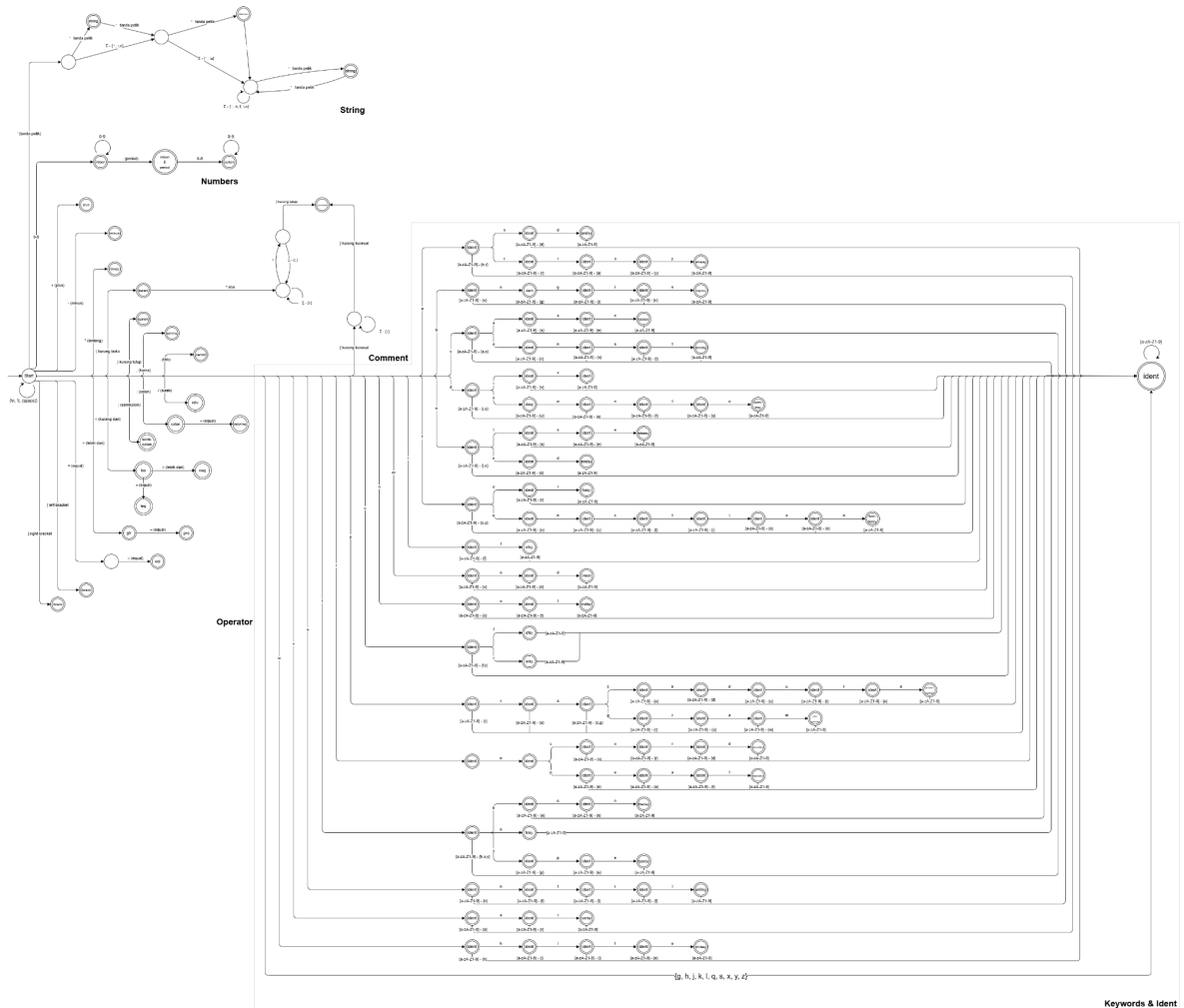
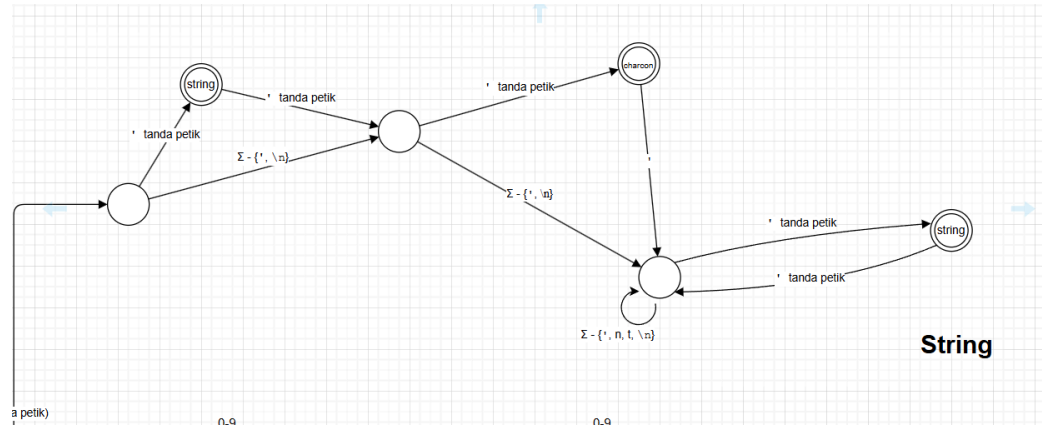


Diagram transisi DFA yang dirancang terdiri dari satu state awal (Start) yang menjadi titik masuk seluruh proses tokenisasi. Dari state Start, DFA membaca karakter pertama dari input dan menentukan jalur transisi yang sesuai. State Start memiliki self-loop untuk karakter  $\{\backslash n, \backslash t, (\text{space})\}$ , sehingga seluruh whitespace diabaikan secara langsung tanpa menghasilkan token apapun. Mekanisme ini memastikan bahwa spasi, tab, dan baris baru antar token tidak ikut terbaca sebagai bagian dari lexeme manapun.

DFA diorganisasikan menjadi beberapa sub-diagram berdasarkan kategori token, yang dijelaskan satu per satu berikut ini.

### A. Sub-Diagram String dan Charcon



Sub-diagram ini menangani token charcon (konstanta karakter tunggal) dan string (sekuens karakter). Proses dimulai ketika state Start membaca karakter ' (tanda petik tunggal), yang kemudian bertransisi ke state awal string/charcon. Dari state tersebut, DFA membaca karakter berikutnya dengan dua kemungkinan jalur:

#### 1. Jalur charcon:

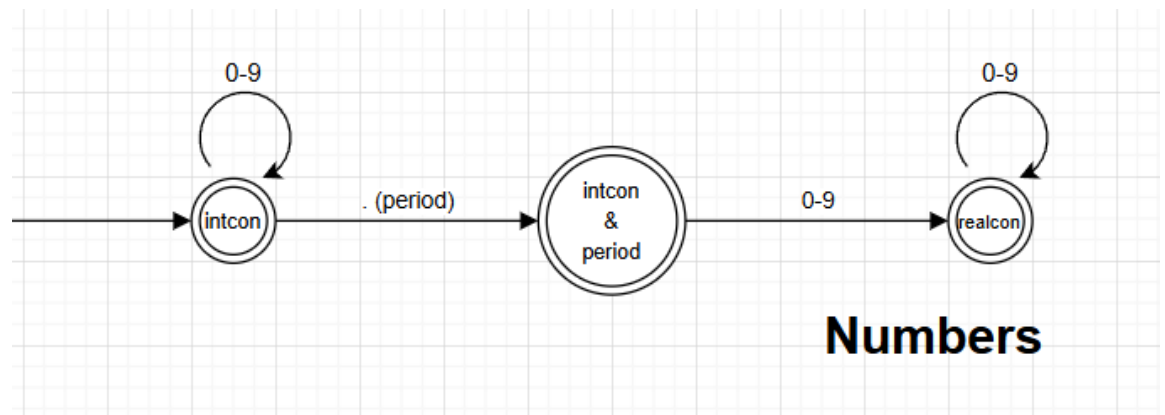
- Karakter pertama di dalam petik adalah sembarang karakter dari  $\Sigma - \{ ', \backslash n \}$  → masuk ke state antara.
- Jika karakter langsung berikutnya adalah tanda petik penutup maka DFA bertransisi ke accepting state charcon, sehingga charcon hanya menerima tepat satu karakter di antara dua tanda petik.

#### 2. Jalur string:

- Dari state antara (setelah karakter pertama terbaca), jika karakter berikutnya bukan tanda petik penutup melainkan karakter lain dari  $\Sigma - \{ ', n, t, \backslash n \}$ , DFA bertransisi ke state string.
- DFA terus berputar di state string selama karakter yang dibaca masih bagian dari  $\Sigma - \{ ', n, t, \backslash n \}$ .
- Ketika tanda petik penutup ' akhirnya ditemukan, maka DFA bertransisi ke accepting state string. sehingga string menerima dua karakter atau lebih di antara dua tanda petik.



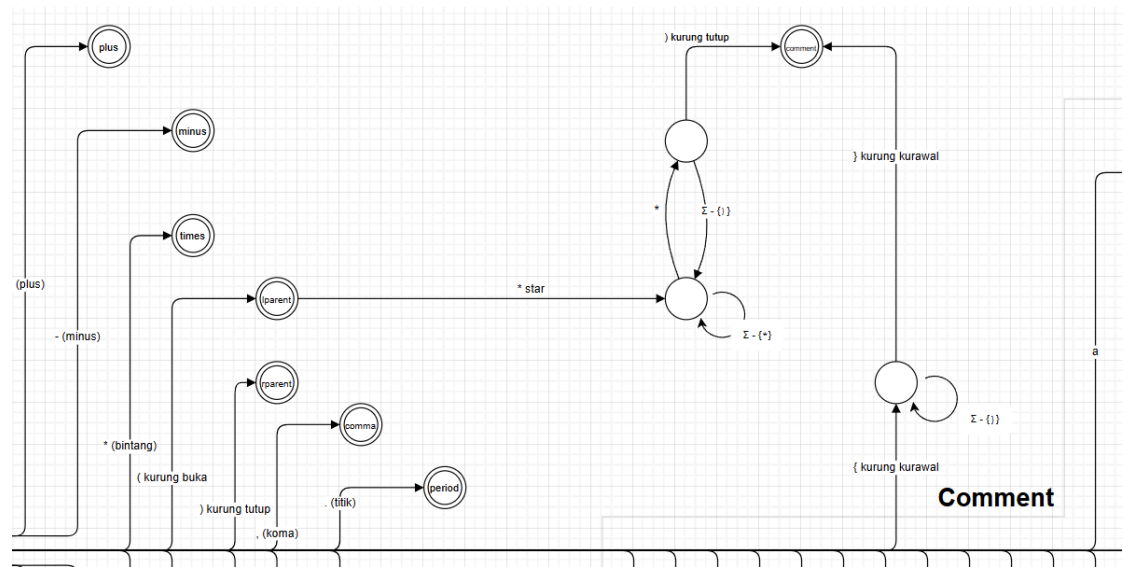
## B. Sub-Diagram Numbers (Intcon dan Realcon)



Sub-diagram ini menangani token angka, yaitu intcon (konstanta integer) dan realcon (konstanta bilangan riil). DFA masuk ke sub-diagram ini ketika state Start membaca digit 0–9.

1. Selama karakter yang dibaca adalah digit 0–9, DFA tetap berada di accepting state intcon dengan self-loop 0–9, menandakan bahwa integer sedang terbentuk dan terus bertambah panjang.
2. Jika ditemukan karakter . (period), DFA bertransisi ke state antara intcon & period. State ini bukan accepting state, DFA belum dapat menghasilkan token karena titik desimal belum dikonfirmasi sebagai pemisah desimal.
3. Dari state intcon & period, jika digit 0–9 berikutnya ditemukan, DFA berpindah ke accepting state realcon dengan self-loop 0–9, menandakan bilangan riil telah terbentuk.

### C. Sub-Diagram Comment



Sub-diagram ini menangani token comment, yaitu teks komentar yang dimulai dengan { atau (\* dan diakhiri dengan } atau \*). Jalur komentar kurung kurawal { ... }:

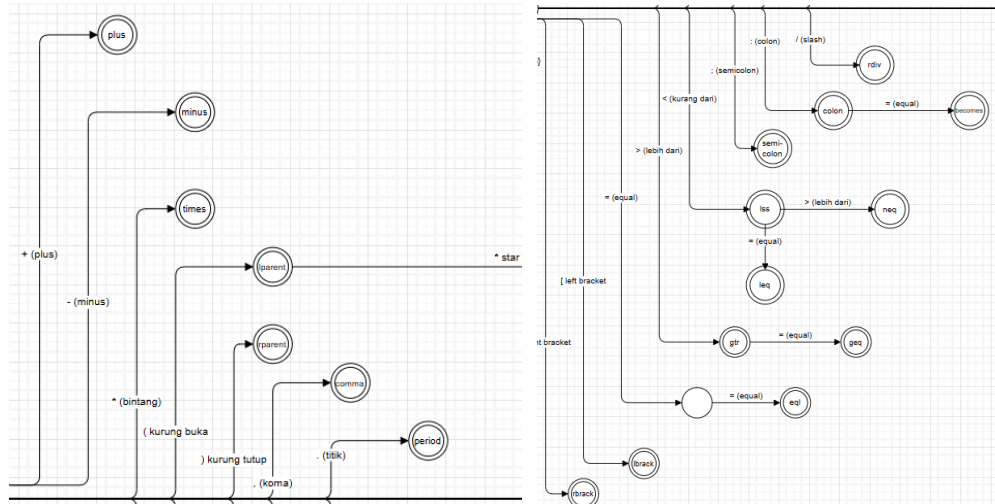
- State Start membaca { lalu masuk ke state antara dalam komentar.
- Selama karakter adalah  $\Sigma - \{\}$  (sembarang karakter kecuali }), DFA tetap di state tersebut dengan self-loop.
- Ketika } ditemukan, maka transisi ke accepting state comment.

Jalur komentar tanda bintang (\* ... \*):

- State Start membaca ( (kurung buka), percabangan: jika karakter berikutnya adalah \* (star), DFA masuk ke jalur komentar; jika bukan, DFA menuju accepting state lparent.
- Di dalam jalur komentar, selama karakter adalah  $\Sigma - \{*\}$  (bukan bintang), DFA berputar di state dalam komentar dengan self-loop.
- Ketika \* ditemukan, maka DFA menuju state penutup komentar.
- Jika karakter berikutnya adalah ) (kurung tutup), transisi ke accepting state comment.
- Jika setelah \* bukan ) melainkan  $\Sigma - \{)\}$ , DFA kembali ke state dalam komentar untuk melanjutkan pembacaan.

## D. Sub-Diagram Operator dan Tanda Baca

Sub-diagram ini menangani seluruh token operator dan tanda baca. Dari state Start, karakter-karakter berikut langsung menuju accepting state masing-masing dalam satu transisi:



| Karakter Input | Accepting State |
|----------------|-----------------|
| +              | plus            |
| -              | minus           |
| * (bintang)    | times           |
| / (slash)      | rdiv            |
| ( kurung buka  | lparent         |
| ) kurung tutup | rparent         |
| , (koma)       | comma           |
| . (titik)      | period          |
| ; (titik koma) | semicolon       |

|                     |        |
|---------------------|--------|
| [ kurung siku kiri  | lbrack |
| ] kurung siku kanan | rbrack |

Beberapa operator memerlukan pembacaan karakter kedua (lookahead) sebelum token dapat ditentukan:

| Karakter Input | Lookahead   | Token yang Dihasilkan |
|----------------|-------------|-----------------------|
| :              | =           | becomes               |
| :              | selain =    | colon                 |
| <              | >           | neq                   |
| <              | =           | leq                   |
| <              | selain >, = | lss                   |
| >              | =           | geq                   |
| >              | selain =    | gtr                   |
| =              | =           | eql                   |

## E. Sub-Diagram Keywords dan Identifier



Sub-diagram ini adalah bagian terbesar dan paling kompleks dari DFA, menangani token keywords (kata kunci) dan ident (identifier). DFA masuk ke sub-diagram ini ketika state Start membaca huruf a–z atau A–Z. Karena bahasa Arion bersifat case-insensitive, setiap transisi pada jalur keyword menerima huruf kecil maupun huruf kapital yang ekuivalen.

Setiap state ident di sepanjang jalur keyword memiliki self-loop [a-zA-Z0-9]. Mekanisme ini mengimplementasikan prinsip Longest Possible Match: jika setelah urutan karakter keyword ditemukan karakter alfanumerik tambahan, DFA tidak berhenti di accepting state keyword, melainkan terus melanjutkan ke accepting state ident.

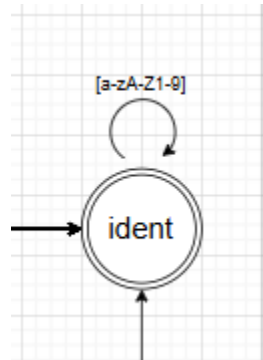
Keywords yang dikenali berdasarkan huruf awal dari state Start:

1. a: a→n→d → andsy ; a→r→r→a→y → arraysy
2. b: b→e→g→i→n → beginsy
3. c: c→a→s→e → casesy ; c→o→n→s→t → constsy
4. d: d→i→v → idiv ; d→o → dosy ; d→o→w→n→t→o → downtosy
5. e: e→l→s→e → elsesy ; e→n→d → endsy
6. f: f→o→r → forsy ; f→u→n→c→t→i→o→n → functionsy
7. i: i→f → ifsy
8. m: m→o→d → imod
9. n: n→o→t → notsy
10. o: o→f → ofsy ; o→r → orsy

11. p:  $p \rightarrow r \rightarrow o \rightarrow c \rightarrow e \rightarrow d \rightarrow u \rightarrow r \rightarrow e \rightarrow$  proceduresy ;  $p \rightarrow r \rightarrow o \rightarrow g \rightarrow r \rightarrow a \rightarrow m \rightarrow$  programsy
12. r:  $r \rightarrow e \rightarrow c \rightarrow o \rightarrow r \rightarrow d \rightarrow$  recordsy ;  $r \rightarrow e \rightarrow p \rightarrow e \rightarrow a \rightarrow t \rightarrow$  repeatsy
13. t:  $t \rightarrow h \rightarrow e \rightarrow n \rightarrow$  thensy ;  $t \rightarrow o \rightarrow$  tosy ;  $t \rightarrow y \rightarrow p \rightarrow e \rightarrow$  typesy
14. u:  $u \rightarrow n \rightarrow t \rightarrow i \rightarrow l \rightarrow$  untilsy
15. v:  $v \rightarrow a \rightarrow r \rightarrow$  varsy
16. w:  $w \rightarrow h \rightarrow i \rightarrow l \rightarrow e \rightarrow$  whilesy

Huruf-huruf yang tidak memulai keyword apapun (g, h, j, k, l, q, s, x, y, z dan ekuivalennya dalam huruf kapital) langsung menuju accepting state ident dengan self-loop [a-zA-Z0-9] untuk membaca sisa identifier.

#### F. Accepting State ident



Di ujung kanan diagram terdapat satu accepting state ident tunggal yang menjadi tujuan dari semua jalur keyword yang tidak cocok maupun dari semua huruf awal non-keyword. State ini memiliki self-loop [a-zA-Z0-9] sehingga seluruh karakter alfanumerik berikutnya terus dibaca dan diakumulasikan sebagai bagian dari identifier yang sama.

## 2.2. Implementasi Program

### A. Modul Klasifikasi Karakter

Modul ini bertanggung jawab untuk mendefinisikan himpunan karakter yang valid dan menyediakan fungsi utilitas (bantuan) untuk mengklasifikasikan karakter tunggal (apakah itu huruf, angka, spasi, atau simbol). Struktur data dari modul ini adalah:

- `struct Alphabet`: yaitu *struct* statis yang membungkus konstanta string (letters, digit, whitespace, dan symbols) dan fungsi-fungsi pengecekan.

| Fungsi/Prosedur                                 | Output  | Keterangan                                                                                                                                                                     |
|-------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>is_letter(char c)</code>                  | boolean | Mengecek apakah karakter adalah huruf alfabet (a-z atau A-Z). Fungsi ini bersifat <i>case-insensitive</i> karena mengubah karakter ke huruf kecil sebelum dicek.               |
| <code>is_digit(char c)</code>                   | boolean | Mengecek apakah karakter adalah angka (0-9).                                                                                                                                   |
| <code>is_whitespace(char c)</code>              | boolean | Mengecek apakah karakter adalah <i>whitespace</i> (spasi, <i>tab</i> , <i>newline</i> <code>\n</code> , atau <i>carriage return</i> <code>\r</code> ).                         |
| <code>is_symbol(char c)</code>                  | boolean | Mengecek apakah karakter termasuk dalam daftar simbol yang diizinkan (seperti <code>+</code> , <code>-</code> , <code>*</code> , <code>&lt;</code> , <code>&gt;</code> , dll). |
| <code>to_lower(const std::string&amp; s)</code> | string  | Fungsi untuk mengubah seluruh huruf dalam sebuah string menjadi huruf kecil ( <i>lowercase</i> ).                                                                              |

## B. Modul Representasi Token

Modul ini mendefinisikan tipe-tipe token yang dapat dikenali oleh Lexer dan struktur data untuk menyimpan informasi token yang ditemukan beserta posisinya di dalam file sumber. Struktur data dari modul ini adalah:

- `enum class TokenType`: merupakan enumerasi yang mendefinisikan semua jenis token yang valid. Terdiri dari tipe data (*INTCON*, *REALCON*, *STRING*), operator matematika/logika (*PLUS*, *MINUS*, *ANDSY*), tanda baca (*LPARENT*, *COMMA*, *SEMICOLON*), hingga *keyword* bawaan bahasa (*PROGRAMSY*, *BEGINSY*, *IFSY*), dan *INVALID* untuk *error*.
- `struct Token`: Struktur yang menyimpan data spesifik sebuah token:
  1. `type (TokenType)`: Jenis token.
  2. `lexeme (std::string)`: String asli dari kode sumber (misal: "123", "begin", "+").
  3. `line_num, col_num (int)`: Posisi baris dan kolom token tersebut.
  4. `consumed (bool)`: Penanda apakah karakter sudah dimasukkan ke *buffer*.

| Fungsi/Prosedur                                                         | Output  | Keterangan                                                                                                                                                                                               |
|-------------------------------------------------------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>consume(char c)</code>                                            | -       | Menambahkan satu karakter ke dalam atribut <code>lexeme</code> (membentuk string) dan mengubah status <code>consumed</code> menjadi <i>true</i> .                                                        |
| <code>is_empty() const</code>                                           | boolean | Mengecek apakah atribut <code>lexeme</code> masih kosong.                                                                                                                                                |
| <code>operator&lt;&lt;(std::ostream&amp; os, const Token&amp; t)</code> | -       | Melakukan <i>overloading</i> pada operator <i>output</i> agar objek token dapat dicetak langsung ke <i>console</i> (misal dengan <code>std::cout &lt;&lt; token;</code> ) dengan format visual yang rapi |



|  |  |                                                                         |
|--|--|-------------------------------------------------------------------------|
|  |  | (contoh: <code>ident(nama_variabel)</code> atau <code>beginsy</code> ). |
|--|--|-------------------------------------------------------------------------|

### C. Modul Pembaca Karakter

Modul ini bertindak sebagai mesin pembaca karakter (Character Machine) yang membaca *file* secara sekuensial (karakter demi karakter). Mesin ini juga melacak posisi kursor (baris dan kolom). Struktur data dari modul ini adalah:

- `class CharMachine`: Memiliki atribut privat seperti `stream_` (`std::ifstream`) untuk membaca *file*, `current_` (karakter saat ini), serta `line_num_` dan `col_num_` untuk pelacakan posisi.

| Fungsi/Prosedur                                           | Output  | Keterangan                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>CharMachine(const std::string&amp; filepath)</code> | -       | <i>Constructor</i> yang membuka <i>file</i> target dan langsung membaca karakter pertamanya. Jika <i>file</i> gagal dibuka, akan melempar <i>exception</i> .                                                                                                                                                                        |
| <code>~CharMachine()</code>                               | -       | <i>Destructor</i> untuk menutup <i>file stream</i> saat objek dihancurkan untuk mencegah <i>memory/resource leak</i> .                                                                                                                                                                                                              |
| <code>advance()</code>                                    | boolean | Membaca karakter selanjutnya dari stream ke variabel <code>current_</code> . Fungsi ini juga mengelola logika posisi (menambah <code>col_num_</code> , atau menambah <code>line_num_</code> jika bertemu karakter <i>newline</i> <code>\n</code> ). Mengembalikan nilai <i>false</i> jika sudah mencapai akhir file ( <i>EOF</i> ). |
| <code>current() const</code>                              | char    | Fungsi <i>getter</i> untuk mengambil karakter yang sedang dibaca saat ini.                                                                                                                                                                                                                                                          |

|                                                                |                      |                                                                                  |
|----------------------------------------------------------------|----------------------|----------------------------------------------------------------------------------|
| <code>col_num() const,</code><br><code>line_num() const</code> | <code>int</code>     | Fungsi <i>getter</i> untuk mengambil informasi posisi kursor pembaca karakter.   |
| <code>eof() const</code>                                       | <code>boolean</code> | Mengecek apakah proses pembacaan sudah mencapai akhir <i>file</i> (End Of File). |

#### D. Modul State Machine

Modul ini mengimplementasikan *Deterministic Finite Automaton* (DFA). Tergantung pada *state* atau status saat ini, mesin akan memutuskan apa yang harus dilakukan ketika menerima karakter baru. Struktur data dari modul ini adalah:

- `enum class State`: Kumpulan *state* dari DFA (contoh: `START`, `IN_IDENT`, `IN_INTCON`, `IN_STRING`, dll).
- `using StateOrToken = std::variant<State, TokenType>`: Tipe data *union* modern (C++17) yang memungkinkan suatu fungsi mengembalikan dua kemungkinan: *State* tujuan selanjutnya, ATAU *Token* final jika pembacaan kata sudah selesai.
- `class Lexer::State`: Kelas yang membungkus logika transisi state. Memiliki array penunjuk fungsi (lambdas) `lookup_` untuk mempercepat proses perpindahan state.

| Fungsi/Prosedur                                      | Output               | Keterangan                                                                                                                                                                                                                                                                       |
|------------------------------------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>State(StateName name, Lexer&amp; lexer)</code> | <code>boolean</code> | Di dalam konstruktor ini, semua logika perpindahan DFA didefinisikan secara <i>inline</i> menggunakan fungsi Lambda C++. Setiap state seperti <code>start</code> , <code>in_quote</code> , <code>in_ident</code> memiliki lambdanya sendiri untuk menangani karakter yang masuk. |
| <code>transition()</code>                            | <code>boolean</code> | Mengeksekusi perpindahan state. Membaca karakter dari mesin karakter, memasukkannya                                                                                                                                                                                              |

|                                   |                  |                                                                                                                                                                            |
|-----------------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                   |                  | ke fungsi lambda yang sesuai di <code>lookup_</code> , dan memperbarui <code>name_</code> (state saat ini) atau memproduksi <i>Token</i> baru ke dalam daftar token.       |
| <code>parse_symbol(char c)</code> | StateOr<br>Token | Fungsi untuk menangani simbol matematika atau tanda baca (termasuk membedakan simbol tunggal seperti <code>&lt;</code> atau simbol ganda seperti <code>&lt;=</code> ).     |
| <code>parse_keyword()</code>      | StateOr<br>Token | Fungsi untuk membedakan apakah kata yang baru saja dibaca merupakan sebuah <i>Keyword</i> bahasa pemrograman (misal: "if", "while") atau sebuah Identifier/Variabel biasa. |
| <code>consumed() const</code>     | boolean          | <i>Getter</i> untuk mengetahui apakah buffer karakter sudah diproses.                                                                                                      |

## E. Modul Lexical Analyzer

Modul ini merupakan kelas *controller* yang menghubungkan CharMachine (pembaca) dengan `Lexer::State` (logika mesin transisi) untuk menghasilkan hasil akhir berupa sekumpulan token. Struktur data dari modul ini adalah:

- `class Lexer`: Memiliki referensi ke CharMachine, vektor dari kelas *Token* untuk menyimpan hasil akhir, dan *pointer* ke objek State saat ini.

| Fungsi/Prosedur                              | Output  | Keterangan                                                                                                              |
|----------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------------------|
| <code>Lexer(CharMachine &amp; reader)</code> | -       | <i>Constructor</i> yang menginisialisasi Lexer, menghubungkannya dengan <i>reader</i> dan menyiapkan DFA di state START |
| <code>read()</code>                          | boolean | Melakukan satu siklus pembacaan: memicu                                                                                 |

|                             |   |                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             |   | <p>fungsi <code>transition()</code> pada objek <i>State</i>. Jika token yang dihasilkan berjenis <code>INVALID</code>, fungsi ini juga bertanggung jawab mencetak pesan <i>error</i> ke <i>console</i> (menyebutkan baris, kolom, dan lexeme yang bermasalah). Fungsi kemudian memanggil <code>advance()</code> pada pembaca karakter.</p> |
| <code>run()</code>          | - | <p>Fungsi utama (<i>entry point</i>) dari objek lexer. Menjalankan perulangan <code>while (read())</code> terus menerus hingga seluruh <i>file</i> selesai dibaca.</p>                                                                                                                                                                     |
| <code>tokens() const</code> | - | <p><i>Getter</i> untuk mengembalikan daftar list (<code>std::vector</code>) seluruh token yang telah berhasil dikenali setelah pemrosesan selesai.</p>                                                                                                                                                                                     |

## BAB 3

### PENGUJIAN

#### 3.1. Hasil Pengujian

##### A. Pengujian ke-1

| 11_downto_if_else.in.txt                                                         |                                                                                                                                                                                                           |
|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input                                                                            | Output                                                                                                                                                                                                    |
| <pre> program P; begin for i downto j do k := 0; if p then q else r; end. </pre> | <pre> programsy ident (P) semicolon beginsy forsy ident (i) downtosy ident (j) dosy ident (k) becomes intcon (0) semicolon ifsy ident (p) thensy ident (q) elsesy ident (r) semicolon endsy period </pre> |
| <b>Keterangan:</b> Sesuai                                                        |                                                                                                                                                                                                           |

##### B. Pengujian ke-2

| 12_bubble_sort.in.txt                |                                           |
|--------------------------------------|-------------------------------------------|
| Input                                | Output                                    |
| <pre> program BubbleSort; var </pre> | <pre> programsy ident (BubbleSort) </pre> |

|                            |                 |
|----------------------------|-----------------|
| i, j, t: integer;          | semicolon       |
| a: array[1..5] of integer; | varsy           |
| begin                      | ident (i)       |
| i := 1;                    | comma           |
| while i <= 5 do            | ident (j)       |
| begin                      | comma           |
| j := 1;                    | ident (t)       |
| while j < 5 do             | colon           |
| begin                      | ident (integer) |
| if a[j] > a[j + 1] then    | semicolon       |
| begin                      | ident (a)       |
| t := a[j];                 | colon           |
| a[j] := a[j + 1];          | arraysy         |
| a[j + 1] := t;             | lbrack          |
| end;                       | intcon (1.)     |
| j := j + 1;                | period          |
| end;                       | intcon (5)      |
| i := i + 1;                | rbrack          |
| end;                       | ofsy            |
| end.                       | ident (integer) |
|                            | semicolon       |
|                            | beginsy         |
|                            | ident (i)       |
|                            | becomes         |
|                            | intcon (1)      |
|                            | semicolon       |
|                            | whilesy         |
|                            | ident (i)       |
|                            | leq             |
|                            | intcon (5)      |
|                            | dosy            |
|                            | beginsy         |
|                            | ident (j)       |
|                            | becomes         |
|                            | intcon (1)      |
|                            | semicolon       |
|                            | whilesy         |
|                            | ident (j)       |
|                            | lss             |
|                            | intcon (5)      |
|                            | dosy            |
|                            | beginsy         |
|                            | ifsy            |
|                            | ident (a)       |
|                            | lbrack          |
|                            | ident (j)       |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | rbrack<br>gtr<br>ident (a)<br>lbrack<br>ident (j)<br>plus<br>intcon (1)<br>rbrack<br>thensy<br>beginsy<br>ident (t)<br>becomes<br>ident (a)<br>lbrack<br>ident (j)<br>rbrack<br>semicolon<br>ident (a)<br>lbrack<br>ident (j)<br>rbrack<br>becomes<br>ident (a)<br>lbrack<br>ident (j)<br>plus<br>intcon (1)<br>rbrack<br>semicolon<br>ident (a)<br>lbrack<br>ident (j)<br>plus<br>intcon (1)<br>rbrack<br>becomes<br>ident (t)<br>semicolon<br>endsy<br>semicolon<br>ident (j)<br>becomes<br>ident (j)<br>plus<br>intcon (1)<br>semicolon |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                           |                                                                                                                                     |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
|                           | endsy<br>semicolon<br>ident (i)<br>becomes<br>ident (i)<br>plus<br>intcon (1)<br>semicolon<br>endsy<br>semicolon<br>endsy<br>period |
| <b>Keterangan:</b> Sesuai |                                                                                                                                     |

### C. Pengujian ke-3

| 13_prime_check.in.txt                                                                                                                                                                                |                                                                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input                                                                                                                                                                                                | Output                                                                                                                                                                                                                                                    |
| <pre> program PrimeCheck; var   n, i, ok: integer; begin   n := 17;   ok := 1;   i := 2;   while i * i &lt;= n do   begin     if n mod i == 0 then       ok := 0;     i := i + 1;   end; end. </pre> | <pre> programsy ident (PrimeCheck) semicolon varsy ident (n) comma ident (i) comma ident (ok) colon ident (integer) semicolon beginsy ident (n) becomes intcon (17) semicolon ident (ok) becomes intcon (1) semicolon ident (i) becomes intcon (2) </pre> |



|                           |                                                                                                                                                                                                                                                                                                                                              |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                           | semicolon<br>whilesy<br>ident (i)<br>times<br>ident (i)<br>leq<br>ident (n)<br>dosy<br>beginsy<br>ifsy<br>ident (n)<br>imod<br>ident (i)<br>eql<br>intcon (0)<br>thensy<br>ident (ok)<br>becomes<br>intcon (0)<br>semicolon<br>ident (i)<br>becomes<br>ident (i)<br>plus<br>intcon (1)<br>semicolon<br>endsy<br>semicolon<br>endsy<br>period |
| <b>Keterangan:</b> Sesuai |                                                                                                                                                                                                                                                                                                                                              |

#### D. Pengujian ke-4

| 14_newton_sqrt.in.txt                                                             |                                                                             |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Input                                                                             | Output                                                                      |
| program NewtonSqrt;<br>var<br>x, g, eps: real;<br>begin<br>x := 2.0;<br>g := 1.0; | programsy<br>ident (NewtonSqrt)<br>semicolon<br>varsy<br>ident (x)<br>comma |

```

eps := 0.0001;
while g * g - x > eps do
begin
    g := (g + x / g) / 2.0;
end;
end.

```

```

ident (g)
comma
ident (eps)
colon
ident (real)
semicolon
beginsy
ident (x)
becomes
realcon (2.0)
semicolon
ident (g)
becomes
realcon (1.0)
semicolon
ident (eps)
becomes
realcon (0.0001)
semicolon
whilesy
ident (g)
times
ident (g)
minus
ident (x)
gtr
ident (eps)
dosy
beginsy
ident (g)
becomes
lparent
ident (g)
plus
ident (x)
rdiv
ident (g)
rparent
rdiv
realcon (2.0)
semicolon
endsy
semicolon
endsy
period

```

**Keterangan:** Sesuai

#### E. Pengujian ke-5

15\_fast\_inv\_sqrt.in.txt

##### Input

```
program FastInvSqrt;  
var  
  x, y, threehalfs: real;  
begin  
  x := 2.25;  
  threehalfs := 1.5;  
  y := x;  
  y := y * (threehalfs - 0.5 * x * y * y *  
y);  
  y := y * (threehalfs - 0.5 * x * y * y *  
y);  
end.
```

##### Output

```
programsy  
ident (FastInvSqrt)  
semicolon  
varsy  
ident (x)  
comma  
ident (y)  
comma  
ident (threehalfs)  
colon  
ident (real)  
semicolon  
beginsy  
ident (x)  
becomes  
realcon (2.25)  
semicolon  
ident (threehalfs)  
becomes  
realcon (1.5)  
semicolon  
ident (y)  
becomes  
ident (x)  
semicolon  
ident (y)  
becomes  
ident (y)  
times  
lparent  
ident (threehalfs)  
minus  
realcon (0.5)  
times  
ident (x)  
times  
ident (y)  
times
```

|                           |                                                                                                                                                                                                                                                                                                     |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                           | ident (y)<br>times<br>ident (y)<br>rparent<br>semicolon<br>ident (y)<br>becomes<br>ident (y)<br>times<br>lparent<br>ident (threehalfs)<br>minus<br>realcon (0.5)<br>times<br>ident (x)<br>times<br>ident (y)<br>times<br>ident (y)<br>times<br>ident (y)<br>rparent<br>semicolon<br>endsy<br>period |
| <b>Keterangan:</b> Sesuai |                                                                                                                                                                                                                                                                                                     |

#### F. Pengujian ke-6

| 06_terminated_string.in.txt       |                                                                                                            |
|-----------------------------------|------------------------------------------------------------------------------------------------------------|
| Input                             | Output                                                                                                     |
| x := 'no closing quote<br>y := 2; | ident (x)<br>becomes<br>unknown ('no closing quote<br>)<br>ident (y)<br>becomes<br>intcon (2)<br>semicolon |
| <b>Keterangan:</b> Sesuai         |                                                                                                            |

### G. Pengujian ke-7

| 07_unclosed_brace_comment.in.txt                               |                                                                                                |
|----------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Input                                                          | Output                                                                                         |
| start; { this brace comment is never closed before end of file | ident (start)<br>semicolon<br>unknown({ this brace comment is never closed before end of file) |
| <b>Keterangan:</b> Sesuai                                      |                                                                                                |

### H. Pengujian ke-8

| 08_unclosed_paren_star_comment.in.txt |                                                                |
|---------------------------------------|----------------------------------------------------------------|
| Input                                 | Output                                                         |
| ok; (* star comment not closed        | ident (ok)<br>semicolon<br>unknown((* star comment not closed) |
| <b>Keterangan:</b> Sesuai             |                                                                |

### I. Pengujian ke-9

| 09_underscore_not_ident.in.txt |                                                                       |
|--------------------------------|-----------------------------------------------------------------------|
| Input                          | Output                                                                |
| x := _notAnIdent;              | ident (x)<br>becomes<br>unknown(_)<br>ident (notAnIdent)<br>semicolon |
| <b>Keterangan:</b> Sesuai      |                                                                       |

### J. Pengujian ke-10

| 10_unknowns_with_blank_lines.in.txt |        |
|-------------------------------------|--------|
| Input                               | Output |

|                           |                          |
|---------------------------|--------------------------|
| @<br><br>#                | unknown(@)<br>unknown(#) |
| <b>Keterangan:</b> Sesuai |                          |

## BAB 4

### KESIMPULAN DAN SARAN

#### 4.1. Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Model *Deterministic Finite Automata* (DFA) terbukti dapat digunakan untuk diimplementasikan sebagai *Lexical Analyzer*. Setiap pola *token* (seperti *identifier*, *keywords*, operator, *string*, dan angka) dapat direpresentasikan ke dalam transisi *state* yang deterministik tanpa ambiguitas.
2. Pemisahan program ke dalam beberapa modul spesifik (Modul Klasifikasi Karakter, Representasi Token, Pembaca Karakter, dan State Machine) membuat kode menjadi lebih terstruktur, mudah dibaca, dan mudah untuk di-*debug* atau dikembangkan lebih lanjut.
3. Implementasi sistem telah berhasil menggunakan prinsip *Longest Possible Match* dan *Rule Priority*. Hal ini dibuktikan dengan kemampuan program membedakan *keyword* bawaan bahasa dan *identifier* yang memiliki huruf awal yang sama.
4. Berdasarkan 10 pengujian yang dilakukan, program Lexer mampu memproses sintaksis program valid, serta mampu menangani dan melaporkan *error* pada *edge-cases* secara tepat..

#### 4.2. Saran

Meskipun *Lexical Analyzer* telah berfungsi dengan baik sesuai spesifikasi, terdapat beberapa ruang pengembangan untuk penyempurnaan di masa depan:

1. Modul DFA dapat diperluas untuk mengenali format tipe data yang lebih kompleks, seperti angka heksadesimal, notasi ilmiah, atau *nested comments*.
2. Program dapat dikembangkan untuk memiliki fitur *error recovery*, di mana Lexer dapat melompati karakter yang salah dan menganalisis sisa *file* tanpa harus berhenti sepenuhnya, serta memberikan pesan *error* yang lebih deskriptif.

## **BAB 5**

### **LAMPIRAN**

#### **5.1. Link Repository**

<https://github.com/Gixgine-budi/MLB-Tubes-IF2224-2026.git>

#### **5.2. Link DFA**

[https://app.diagrams.net/#G1mQIBXhcHHuxeH3iKnkEMrnb2LsDPeQGO#%7B%22pageId%22%3A%22wVceDYj46z\\_NUhlbeJMC%22%7D](https://app.diagrams.net/#G1mQIBXhcHHuxeH3iKnkEMrnb2LsDPeQGO#%7B%22pageId%22%3A%22wVceDYj46z_NUhlbeJMC%22%7D)



## REFERENSI

- <https://lcc.s3.amazonaws.com/book/pdf/06lexical-analysis.pdf>
- <https://symbolaris.com/course/Compilers/06-lex.pdf>
- <https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/lectures/01/Slides01.pdf>